

React Hooksを使いこなせ！

初心者でも簡単。ポートフォリオ開発で実践するReact講座

田中博悠 / UniSchool

今日のゴール

- React Hooksが「何をするためのもの」か理解する
- 主要なHooksを一通り知る（`useState` ～ `useContext` まで）
- 実務でどう使われるか / どこでハマりやすいかまで知る
- React Router DOMでページ遷移を実装できるようになる
- 配布したサンプルポートフォリオにHooksを実装して完成させる

このスライドは座学パート。実装は `sandbox/` フォルダのTODOコメントを埋めながら進めます。

アジェンダ

1. なぜHooksが生まれたのか
2. `useState` — 状態を持つ
3. `useEffect` — 副作用を扱う
4. `useMemo` / `useCallback` — 計算をメモ化する
5. `useRef` — DOMや値を保持する
6. `useReducer` — 複雑な状態をまとめる
7. `createContext` / `useContext` — 状態を共有する
8. カスタムフック — 自分だけのHookを作る
9. React Router DOM — ページを分割する
10. 演習：サンプルポートフォリオを完成させよう

各章、基本 → 実務での使いどころ → よくある落とし穴 の順で見っていきます。

1. なぜHooksが生まれたのか

Before Hooks (Class Component時代)

- 状態管理は `this.state` / `this.setState`
- ライフサイクルは `componentDidMount` などのメソッドで分散
- ロジックの再利用が難しい (HOCやRender Propsで無理やり共有)

After Hooks (2019年～)

- 関数コンポーネントのままstateやライフサイクルを扱える
- 関連するロジックを1箇所にまとめられる
- カスタムフックでロジックを部品化・再利用できる

実務ではもうClass Componentで新規実装することはほぼありません。既存コードの保守で読む機会があるくらいです。

2. useState — 状態を持つ

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Count: {count}
    </button>
  );
}
```

- `useState(初期値)` → `[現在の値, 更新関数]` のペアを返す
- `setCount` を呼ぶと再レンダリングが起きる
- 更新関数には値 or 「前の値を受け取る関数」を渡せる

useState の注意点

```
// NG: 連続で呼んでも1回分しか反映されないことがある
```

```
setCount(count + 1);
```

```
setCount(count + 1);
```

```
// OK: 関数形式で「前の値」を確実に受け取る
```

```
setCount((prev) => prev + 1);
```

```
setCount((prev) => prev + 1);
```

Stateは直接書き換えない！ `state.push(...)` ではなく `setState([...state, item])` のように新しい値を作って渡す。

useState 実務での使いどころ

ユースケース	例
フォーム入力	<code>const [email, setEmail] = useState("")</code>
トグルUI	モーダルの開閉、アコーディオン、タブ切り替え
APIの状態管理	<code>data</code> / <code>isLoading</code> / <code>error</code> の3点セット
一時的なUI状態	ホバー中フラグ、選択中の行、ページ番号

```
// 関連する値は1つのオブジェクトにまとめてもOK
const [form, setForm] = useState({ name: "", email: "" });
setForm((prev) => ({ ...prev, email: "new@example.com" }));
```

オブジェクトstateを更新するときは `...prev` でコピーを忘れると、更新していないフィールドが消えてしまう。

3. `useEffect` — 副作用を扱う

「副作用」＝レンダー以外でやりたいこと（通信・タイマー・DOM操作など）

```
useEffect(() => {  
  console.log("マウントされた or 依存値が変わった");  
  
  return () => {  
    console.log("クリーンアップ（アンマウント前 or 再実行前）");  
  };  
}, [依存配列]);
```

依存配列	実行タイミング
省略	毎レンダー後
<code>[]</code>	初回マウント時のみ
<code>[a, b]</code>	初回 + a か b が変わった時

useEffect の実例：データ取得

```
function ProjectList() {  
  const [projects, setProjects] = useState([]);  
  const [isLoading, setIsLoading] = useState(true);  
  
  useEffect(() => {  
    const timer = setTimeout(() => {  
      setProjects(projectsData);  
      setIsLoading(false);  
    }, 600);  
  
    return () => clearTimeout(timer); // クリーンアップ  
  }, []); // 初回のみ実行  
  
  if (isLoading) return <Loading />;  
  return <List items={projects} />;  
}
```

sandbox の `useProjects.ts` はまさにこの形を実装する演習です。

useEffect 実務での使いどころ

- データ取得（API呼び出し、ページ遷移時の再取得）
- 外部イベントの購読/解除（`window.addEventListener("resize", ...)`）
- 外部システムとの同期（`document.title` の変更、WebSocket接続）
- タイマー処理（`setInterval` でのポーリング、自動保存）

```
useEffect(() => {  
  const handleResize = () => setWidth(window.innerWidth);  
  window.addEventListener("resize", handleResize);  
  
  return () => window.removeEventListener("resize", handleResize);  
}, []);
```

「登録したら必ず解除する」がセット。addしたらremove、setしたらclearを書く癖をつける。

useEffect よくある落とし穴①：無限ループ

```
// NG: オブジェクト/配列を依存配列に入れると、
// 毎レンダーで「新しい参照」になり、無限に発火する
const options = { limit: 10 };
useEffect(() => {
  fetchData(options);
}, [options]); // ← 毎回 options !== 前回の options

// OK: プリミティブな値だけを依存配列に入れる
useEffect(() => {
  fetchData({ limit });
}, [limit]);
```

「依存配列を減らしたくてESLintの警告を無視する」のは大体バグの温床。警告が出たら「本当にこの値を無視していいか」を考える。

useEffect よくある落とし穴②：レースコンディション

```
useEffect(() => {  
  let ignore = false;  
  
  fetchUser(userId).then((data) => {  
    if (!ignore) setUser(data); // 古いリクエストの結果を無視  
  });  
  
  return () => {  
    ignore = true; // userIdが変わったら前のリクエストは捨てる  
  };  
}, [userId]);
```

- `userId` が素早く切り替わると、後から呼んだリクエストが先に返ることがある
- クリーンアップで「もう使わないフラグ」を立てて古い結果を捨てるのが定石
- 実務では `AbortController` や React Query / SWR がこれを肩代わりしてくれる

4. useMemo — 計算をメモ化する

```
const filteredProjects = useMemo(  
  () => projects.filter((p) => p.title.includes(searchText)),  
  [projects, searchText] // これらが変わった時だけ再計算  
);
```

- 重い計算・配列操作を依存値が変わった時だけやり直す
- 依存配列が変わらなければ、前回の計算結果をそのまま再利用

useCallback — 関数をメモ化する

```
const handleSearchChange = useCallback((value) => {  
  setSearchText(value);  
}, []); // 依存配列が変わらなければ同じ関数を使い回す
```

- `useMemo` の「関数版」
- 子コンポーネントに関数をpropsで渡すとき、
 毎回新しい関数が作られて無駄な再レンダリングが起きるのを防げる
- `React.memo` とセットで使うと効果を発揮する

useMemo / useCallback 実務での考え方

最初から全部につけるのはNG。React公式も「まず測ってから最適化する」ことを推奨している。軽い計算にまで使うと、コードが複雑になるだけで逆に遅くなることもある。

本当に必要になる代表的な場面

場面	理由
配列の filter/sort/reduce が重い or 頻繁	毎レンダー計算を避けたい
React.memo した子コンポーネントに関数/オブジェクトを渡す	参照が変わると memo が効かない
Context の value に渡すオブジェクト	毎回新しい参照だと全consumerが再レンダー
他のHookの依存配列に渡す値	参照の安定化が必要

5. useRef — DOMや値を保持する

```
function ContactForm() {  
  const nameInputRef = useRef(null);  
  
  useEffect(() => {  
    nameInputRef.current?.focus(); // マウント時にフォーカス  
  }, []);  
  
  return <input ref={nameInputRef} name="name" />;  
}
```

- `.current` に値を持たせられる「箱」
- 値を変えても再レンダリングされない（`useState`との一番の違い）
- DOM要素への参照 / タイマーID / 前回の値の保持などに使う

useRef 実務での使いどころ

```
// 1. setInterval / setTimeout のIDを保持する
const timerRef = useRef(null);
timerRef.current = setInterval(tick, 1000);
// ...後で clearInterval(timerRef.current)

// 2. 「前回の値」を覚えておく (usePrevious パターン)
function usePrevious(value) {
  const ref = useRef();
  useEffect(() => {
    ref.current = value;
  });
  return ref.current;
}
```

レンダー中に `ref.current` を読み書きしない（表示に反映されないのに値だけ変わり、バグの温床になる）。基本は「イベントハンドラの中」「useEffectの中」で触る。

6. `useReducer` — 複雑な状態をまとめる

```
function likeReducer(state, action) {  
  switch (action.type) {  
    case "toggle":  
      return { liked: !state.liked, count: state.count + (state.liked ? -1 : 1) };  
    default:  
      return state;  
  }  
}  
  
function LikeButton() {  
  const [state, dispatch] = useReducer(likeReducer, { liked: false, count: 42 });  
  
  return <button onClick={() => dispatch({ type: "toggle" })}>{state.count}</button>;  
}
```

- 複数の値が連動して変わる状態を1つのreducer関数にまとめられる
- `useState` をいくつも並べるより、更新ロジックが1箇所にまとまり見通しが良い

useReducer を使うべきタイミング

こういう時は <code>useState</code> を並べるより <code>useReducer</code>	具体例
複数のstateが同時に・連動して変わる	フォームの「入力値」と「エラー」と「送信中」
次のstateが前のstateとactionから決まる	カート（追加・削除・数量変更）
状態遷移が多い	ウィザード形式のステップ管理

迷ったら最初は `useState` でOK。「更新ロジックがあちこちに散らばって辛い」と感じたら `useReducer` への切り替えを検討する、で十分。

7. createContext / useContext

状態をPropsのバケツリレーなしで下の階層まで届ける仕組み

```
// 1. Contextを作る
const ThemeContext = createContext({ theme: "light", toggleTheme: () => {} });

// 2. Providerで囲んで値を配る
<ThemeContext.Provider value={{ theme, toggleTheme }}>
  <App />
</ThemeContext.Provider>

// 3. 好きな深さのコンポーネントから読む
const { theme, toggleTheme } = useContext(ThemeContext);
```

Context を使うべき時 / 使わなくていい時

使うべき	使わなくていい
テーマ（ダーク/ライト）	ボタンの中だけで完結するstate
ログイン中のユーザー情報	フォーム1個の入力値
言語設定（i18n）	親から子1階層だけで渡せる値

Contextは「グローバル変数」に近い便利さと危険さがある。なんでもContextに入れるとデータの流れるが追いに
くくなる。

Context のよくある落とし穴：再レンダリング

```
// NG: 毎レンダーで value が新しいオブジェクトになり、  
// 全consumerが無駄に再レンダリングされる  
<ThemeContext.Provider value={{ theme, toggleTheme }}>  
  
// OK: useMemoでvalueを安定化する  
const value = useMemo(() => ({ theme, toggleTheme }), [theme]);  
<ThemeContext.Provider value={value}>
```

実務では「頻繁に変わる値」と「あまり変わらない値」でContextを分けることも多い（例: `AuthContext` と `AuthDispatchContext` を分離）。

8. カスタムフック — 自分だけのHookを作る

```
function useProjects() {  
  const [projects, setProjects] = useState([]);  
  const [isLoading, setIsLoading] = useState(true);  
  
  useEffect(() => {  
    const timer = setTimeout(() => {  
      setProjects(projectsData);  
      setIsLoading(false);  
    }, 600);  
    return () => clearTimeout(timer);  
  }, []);  
  
  return { projects, isLoading };  
}
```

- `use` から始まる**ただの関数**。中で他のHooksを呼べる
- 「`useState + useEffect`」のセットをコンポーネントの外に切り出せる
- 複数のコンポーネントで同じロジックを使い回せる

実務でよく使うカスタムフック集

フック名	用途
<code>useDebounce(value, delay)</code>	検索入力などの発火を遅らせる
<code>useLocalStorage(key, initial)</code>	stateをlocalStorageと同期する
<code>useMediaQuery(query)</code>	画面幅に応じてUIを出し分ける
<code>useOnClickOutside(ref, handler)</code>	ドロップダウンの外側クリックで閉じる

```
function useDebounce(value, delay) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const timer = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(timer);
  }, [value, delay]);
  return debounced;
}
```


Hooksのルール（超重要）

1. トップレベルでのみ呼ぶ
→ if文やfor文、ネストした関数の中で呼ばない
2. Reactの関数コンポーネント / カスタムフックの中でのみ呼ぶ
→ 普通のJS関数やイベントハンドラの中では呼ばない

```
// NG
if (isLoggedIn) {
  const [name, setName] = useState("");
}

// OK
const [name, setName] = useState("");
if (isLoggedIn) {
  // ...
}
```

実務では `eslint-plugin-react-hooks` を必ず導入する。このルール違反や依存配列の書き忘れをリアルタイムで警告してくれる。

9. React Router DOM — ページを分割する

```
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";

<BrowserRouter>
  <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/projects" element={<Projects />} />
    <Route path="/projects/:id" element={<ProjectDetail />} />
  </Routes>
</BrowserRouter>

<Link to="/projects">Projects</Link>
```

- SPA（1枚のHTML）のまま、URLごとに表示するコンポーネントを切り替える
- `Link` / `NavLink` でページ遷移（ブラウザリロードなし）

React Router のHooks

```
import { useParams, useNavigate } from "react-router-dom";

function ProjectDetail() {
  const { id } = useParams();          // URLの :id 部分を取得
  const navigate = useNavigate();      // JSからページ遷移させる

  const project = projects.find((p) => p.id === id);

  return (
    <div>
      <button onClick={() => navigate(-1)}>- 戻る</button>
      <h1>{project?.title}</h1>
    </div>
  );
}
```

`/projects/task-board` にアクセスすると `id === "task-board"` が取れる。

React Router 実務Tips

```
// クエリパラメータ (?tag=react) を扱う
const [searchParams, setSearchParams] = useSearchParams();
const tag = searchParams.get("tag");

// どのpathにも一致しない = 404ページ
<Route path="*" element={<NotFound />} />

// ログインしていないと弾く「認証ガード」の基本形
function RequireAuth({ children }) {
  const { user } = useContext(AuthContext);
  return user ? children : <Navigate to="/login" replace />;
}
```

本格的なアプリでは、ページ単位で `React.lazy` + `Suspense` を組み合わせて コード分割（初期表示を軽くする）するのも定番のテクニック。

10. 演習：サンプルポートフォリオを完成させよう

sandbox/ に Vite + React + TypeScript + Tailwind CSS のプロジェクトを用意済み。
デザインは完成済み、Hooksのロジック部分だけ TODO コメントになっています。

ファイル	実装するHook
context/ThemeContext.tsx	useState / useEffect
components/ThemeToggle.tsx	useContext
hooks/useProjects.ts	useState / useEffect （カスタムフック）
pages/Projects.tsx	useState / useMemo
pages/Home.tsx	useMemo
pages/ProjectDetail.tsx	useParams / useNavigate
components/LikeButton.tsx	useReducer （ボーナス）
pages/Contact.tsx	useState / useRef

進め方

```
cd sandbox
npm install
npm run dev
```

1. まず今のデザインをブラウザで一通り触ってみる
2. `README.md` のTODO一覧を上から順に開く
3. コード内のTODOコメントに書かれたヒントを参考に実装する
4. 実装したら必ずブラウザで動作確認する（見た目は変わらないはず！）

詰まったら、このスライドの該当セクションに戻って読み返そう。

まとめ

- Hooksは「関数コンポーネントに状態やライフサイクルを持たせる仕組み」
- `useState` / `useEffect` が基本、`useMemo` / `useCallback` / `useRef` / `useReducer` は最適化・特定用途
- `createContext` + `useContext` でProps地獄から解放される
- カスタムフックでロジックを部品化できる
- React Router DOMでページ単位のアプリが作れる

次のステップ（実務でさらに使うもの）

これを学んだら	次はこれ
<code>useEffect</code> でのデータ取得	React Query / SWR（キャッシュ・再取得を肩代わり）
<code>useReducer</code> + Context	Zustand / Redux Toolkit（大規模な状態管理）
フォームの <code>useState</code> 地獄	React Hook Form / Zod によるバリデーション
ESLintの手動運用	<code>eslint-plugin-react-hooks</code> を必ず導入

お疲れ様でした！ 実装、頑張ってください 🎉